Relocation Within Containers

Enabling trivial relocation for std::vector

Document #:	P2959R0
Date:	2023-10-15
Project:	Programming Language C++
Audience:	Library Evolution Working Group

Reply-to:	Alisdair Meredith <ameredith1@bloomberg.net>

Contents

1 Abstract

The *Cpp17ContainerRequirements*, in conjunction with `std::allocator_traits`, are intended to support a wide variety of allocator behavior. Much of the flexibility of `allocator_traits` was modeled on support that would be needed for stateful allocators like `std::pmr::polymorphic_allocator`. This paper examines how the current requirements and traits fall short when containers must internally relocate elements, and proposes a fix in the form of a new optional customization point, `allocator_traits::relocate`.

2 Revision history

R0: October 2023 (pre-Kona mailing)

— Initial draft of this paper.

3 Introduction

The standard library informally has two different kinds of containers, which we will call *node-based containers* and *block-based containers* in this paper. A node-based container typically manages each element in a separately allocated node, that contains additional bookkeeping information such as pointers to adjacent nodes in that data structure; examples include `std::list` and `std::map`. A *block-based* container manages multiple elements in one or more blocks of contiguously allocated memory; examples include `std::vector` and `std::deque`.

As additional vocabulary we will talk about *elements* denoting the objects owned and managed by a container, and *values* that are the salient state of objects (elements) stored in a container. Containers manipulate elements, while they expose the values of those elements to the outside world, such as through iterators for the Standard Library algorithms. By inference, algorithms, views, and ranges operate on values, not elements, unless they also manage the lifetime of the objects they are supplied.

The distinction between element and value is typically academic in the majority of cases. This paper addresses those occasions where the difference matters, and how well, or badly, the standard library responds. In particular, that distinction becomes a significant concern when seeking to apply optimizations from the core language proposal for *trivial relocatability* ([P2786R2]).

The second goal of this paper serves as an adjunct to [P2786R2], extending the Standard Library to provide better support for optimizing the use of trivially relocatable types. The two goals are placed in the same paper to best handle their interaction in the semantics and optimization of block-based containers. It would be simple to move the pure library extensions of the second goal into a separate paper if preferred.

4 Motivating Examples

We will use `std::vector` as a familiar proxy for block-based containers, and `std::list` as a proxy for node-based sequence containers. Below are a couple of motivating example programs using `tuple<int &>` as the element type, which will expose the subtle, but important, differences between an element and a value.

When constructed `tuple<int &>` binds its reference member to the same object as the constructor argument; however on assignment `tuple<int &>` assigns through its original reference rather than rebinding to the argument. This is an intentional feature of the `tuple` API to support the `std::tie` interface[1].

The current behavior when replacing elements in a vector is to use assignment whereas under our proposal this specific example would effectively rebind the references as-if by construction.

4.1 `list` vs `vector`

Demonstrating inconsistent behavior between containers. `std::list` and `std::vector` will give different results on the last statement below for an otherwise identical program. We believe that difference in behavior is surprising other than in hindsight, and suggest that `std::vector` should behave the same as `std::list` in C++26, giving consistent but backwards-incompatible behavior.

std::list

```
#include <iostream>
#include <tuple>
#include <type_traits>
#include <list>

static_assert(std::is_move_assignable_v<int &>);
static_assert(std::is_move_assignable_v<std::tuple<int
        &>>);

int main() {
    int a = 1;
    int b = 2;
    int c = 3;

    std::list<std::tuple<int &>> x;
    x.emplace_back(a);
    x.emplace_back(b);
    x.emplace_back(c);

    std::cout << "a:\t" << a << "\n"
        // 1
              << "b:\t" << b << "\n"
        // 2
              << "c:\t" << c << "\n";
        // 3


    auto const mid = std::next(x.begin());

    std::cout << "x[0]:\t" << std::get<0>(x.front())
        << "\n"    // 1
              << "x[1]:\t" << std::get<0>(*mid)
        << "\n"    // 2
              << "x[2]:\t" << std::get<0>(x.back())
        << "\n";   // 3

    x.erase(mid);

    std::cout << "x[0]:\t" << std::get<0>(x.front())
        << "\n"    // 1
              << "x[1]:\t" << std::get<0>(x.back())
        << "\n";   // 3

    b = 4;
    c = 5;

    std::cout << "x[0]:\t" << std::get<0>(x.front())
        << "\n"    // 1
              << "x[1]:\t" << std::get<0>(x.back())
        << "\n";   // 5
}
```

std::vector

```
#include <iostream>
#include <tuple>
#include <type_traits>
#include <vector>

static_assert(std::is_move_assignable_v<int &>);
static_assert(std::is_move_assignable_v<std::tuple<int
        &>>);

int main() {
    int a = 1;
    int b = 2;
    int c = 3;

    std::vector<std::tuple<int &>> x;
    x.emplace_back(a);
    x.emplace_back(b);
    x.emplace_back(c);

    std::cout << "a:\t" << a << "\n"
        // 1
              << "b:\t" << b << "\n"
        // 2
              << "c:\t" << c << "\n";
        // 3


    auto const mid = std::next(x.begin());

    std::cout << "x[0]:\t" << std::get<0>(x.front())
        << "\n"    // 1
              << "x[1]:\t" << std::get<0>(*mid)
        << "\n"    // 2
              << "x[2]:\t" << std::get<0>(x.back())
        << "\n";   // 3

    x.erase(mid);

    std::cout << "x[0]:\t" << std::get<0>(x.front())
        << "\n"    // 1
              << "x[1]:\t" << std::get<0>(x.back())
        << "\n";   // 3

    b = 4;
    c = 5;

    std::cout << "x[0]:\t" << std::get<0>(x.front())
        << "\n"    // 1
              << "x[1]:\t" << std::get<0>(x.back())
        << "\n";   // 4
}
```

§ 4.2 `vector` vs `vector`

This example demonstrates inconsistent behavior for the same container performing the same operations at runtime. As the current specification for block-based containers does not preserve elements, very different results are produced when acting on the same set of values held by a vector, depending only on non-value state of the vector itself. This surprising behavior arises as `vector` has different behavior when inserting at capacity, compared to typical insertions that do not require reallocation.

To emphasize we that we are acting on the same object with the same set of values, the example uses lambda objects with reference captures to guarantee that all the operations are happening in the same order on the same elements.

First, we reserve up to a capacity of four, and check that the exact requested capacity is respected. Then we iterate twice: on each pass, we first clear the vector and fill it with four identical items; then we insert a different item into the middle of the vector, at position two; finally we log the state of the vector, writing the value of each element, and then name of the variable each element is bound to.

Therefore the first run through the sequence of lambda operations will relocate elements into a new buffer as the `vector` must expand, while the second run through the lambdas will operate purely within the existing buffer.

```cpp
#include <cassert>
#include <iostream>
#include <tuple>
#include <vector>

int main() {
    int a = 1;
    int b = 2;

    using element = std::tuple<int &>;
    std::vector<element> v;
    v.reserve(4);
    assert(4 == v.capacity());

    auto fill = [&](int & i ) {
        v.clear();
        for (int j = 0; j != 4; ++j) {
            v.emplace_back(i);
        }
    };

    auto inject = [&](int & i) { v.emplace(v.begin() + 2, i); };

    auto report = [&] {
        for (auto& j : v) {
            std::cout << std::get<0>(j);
        }

        for (auto& j : v) {
           if (&a == &std::get<0>(j)) {
                std::cout << 'a';
            }
            else if (&b == &std::get<0>(j)) {
                std::cout << 'b';
            }
            else {
                std::cout << '?';
            }
        }

        std::cout << '\n';
    };

    for (int dummy : {1, 2}) {
        fill(a);
        inject(b);
        report();
    }
}
```

How well can you predict the answer without looking ahead[2]?

Did you predict the first iteration would report exactly one reference to `b` in the middle of the vector? Did you predict the second iteration would contain no references to `b`? Did you predict the final state would contain values that all appear equal to `b`, as `a` is overwritten by `insert` on the second iteration of the loop?

Take note again, that in both iterations the initial value of the vector before the `emplace` in `inject` is exactly four references to `a`; the different behaviors at runtime are entirely due to the `vector` being at capacity for an insertion, or not.

In practice, such issues are much harder to track down outside a test program that has been specifically designed to demonstrate the problem. How well do you think the typical non-committee developer would predict this behavior without the warm-up of a paper describing the specific concern? I will admit that as paper author, the part assigning `a` to `b` caught me out even as I was writing this example for the paper!

§ 5 Related LWG issues

— see also [LWG3758], trying to make the problem worse!

— see also [LWG2158]. Conditional copy/move in `std::vector`

— see also [LWG1102], `std::vector`'s reallocation policy still unclear

— drive-by fix [LWG3308], on `erase` not invalidating iterators unless elements actually erased

6 Current Specification

To understand the concerns we will raise, we should first ensure we understand exactly what the standard specifies.

6.1 Elements are created using `allocator_traits::construct`

Internal relocation, e.g., to a new `vector` buffer, must use `allocator_traits::construct` which can select a potentially-throwing form, i.e., `noexcept(false)`, rather than the typically non-throwing move constructor.

Designed to work with `pmr` data structures that assume all allocators are the same, as managed by the `construct` calls — but must go through run-time test for allocators every time, as `allocator_traits`, through the assignment operators that we must delegate to, cannot make that assumption.

A vendor hand-code a partial specialization of `pmr` containers to apply the internal knowledge, much like Bloomberg BDE, but that defeats the point of a primary template that is allocator-aware and should just Do The Right Thing, rather than defer to vendor QoI and difficult proof of *as-if* rules taking advantage of assumptions of allocator-equality. Note that if we cannot prove the *as-if* constraint to our satisfaction, then such an optimization is also non-conforming.

6.2 Elements are replaced using move-assignment

When a block-container moves elements to restore contiguity after an `erase`, or when moving elements within existing capacity to support an `insert`, the container requirements specify that existing elements are replaced through move assignment, rather than through destruction followed by move construction.

Node-based containers simply rearrange their nodes without touching any existing elements.

As we shall see below, there are a set of types where this inconsistency for internal rearrangement of elements has an observable difference of behavior that is not only surprising, but often counterproductive in those cases.

7 Concerns

There are several concerns raised by the current library specification, that become magnified once we consider applying trivial relocation optimizations. The concerns arise from two distinct sources: 1) from a container's element type, and 2) from a container's allocator type.

7.1 Replacing elements by move-assignment rather than move-construction

When move-assignment for an element type produces a different state to destruction followed by move-construction, block containers behave very differently to node-based containers, that merely move the nodes rather than the elements. This behavior is very clearly specified, not undefined behavior or some other failure on the users' part to supply a good type; the end result is predictable from the inputs in every case, even if it may be surprising.

For example, consider the two motivating examples at the start of this document, which use nothing but standard types.

For another example, if we erase an element from the middle of a container, `std::vector<std::pmr::string>>` behaves very differently to `std::list<std::pmr::string>>`; the `list` moves nodes to relocate elements, preserving allocators *and* values of the stored strings, while the `vector` moves *values*, resulting in reallocation, and the moved values landing in different *elements*.

We would like `vector` to behave more like `list` for all of these examples.

7.2 Allocators may care about element identity

If an allocator customizes the `conatruct` function, it should also customize the `destroy` function. Internal relocation by assignment might then violate allocator expectations for `destroy`, especially if the identity (i.e., the address) of the constructed elements matters.

For brevity, we will defer a simple example of this to the case of supporting trivial relocation (per [P2786R2]. In practice, containers will traffic in the correct sequence of paired `construct` and `destruct` calls for objects created at those addresses if use a simple example such as telemetry to match `construct` calls to `destroy` calls — to properly demonstrate a dependence on

object identity we need a stronger coupling between the object and the allocator, such as logging specific operations. However, attempts to provide an example that was compelling rather than artificial proved tricky

For reference, this is the kind of allocator we are thinking about that becomes compelling only in the presence of trivial optimizations. Note that this example assumes C++20[3].

```
template <typename Element>
struct Telemark : std::allocator<Element> {
   std::set<void *> & d_owned;

   using alloctor::allocator;

   Telemark(std::set<void *> &telemetry) : d_owned{telemetry} {}

   template<class T, class... Args>
   void construct(T* p, Args&&... args) {
      if (!d_owned.emplace(p).second) {
         std::cout << "Overwrite at " << (void *)p << std::endl;
      }

      ::new((void*)p) T(std::forward<Args...>(args...));
   }


   template <class T>
   void destroy(T* p) {
      if (!d_owned.erase(p)) {
         std::cout << "Bad destroy at " << (void *)p << std::endl;
      }
   }
};
```

7.3 Strong exception safety guarantee may be broken

One of the cornerstones of the C++ Standard Library contracts is the notion of the basic and the strong exception safety guarantees (although they are never called that in the standard). The basic guarantee simply promises no resource leaks and retains object invariants if an exception is thrown by a function, whereas the strong exception safety guarantee strengthens the basic by additionally promising that if an exception propagates from a function, then no side effects will occur.

Several containers offer the strong exception safety guarantee when inserting at the end of that container, including `std::vector`. In the case of `vector` the only circumstances under which the strong exception safety guarantee is reduced to the basic are: * the element type does not have a `noexcept` move constructor, and * the element type is not copy constructible

There is no leeway given for the `allocator_traits::construct` function itself throwing, so the following program should be a simple demonstration of the strong exception safety guarantee when the allocator throws on its 8th construction.

In practice, a library can observe that the default implementation of `allocator_traits::construct` simply calls placement new, and can throw only if the called constructor can throw — so optimizing by directly calling the move constructor is a correct implementation for as-if behavior so long the container's allocator does not provide a `construct` call; `std::allocator` relies on the default `allocator_traits::construct` implementation. Once control is transferred to the allocator function though, the assumption no longer holds, even if the `construct` call is marked `noexcept`; the allocator may want to store additional information such as the address of constructed elements, which would be lost if the move constructor is called directly. A `noexcept allocator::construct` would still allow an implementation to avoid the cost of making a redundant copy to roll back.

Note that correcting the current implementations to honor the strong exception safety guarantee in this case could be an extreme pessimization for allocators that are not `std::allocator`. For example, `std::pmr::vector<std::pmr::string>>` would be required to copy its strings on a vector reallocation rather than simply moving them as today, unless the library vendor provides a partial specialization for `std::vector<T, std::pmr::polymorphic_allocator<T>>` to exploit the knowledge that moving `pmr` elements is safe due to semantic guarantees provided by the contract of `pmr::polymorphic_allocator` and enforced by the `constuct` calls.

This paper will propose a more general solution than partially specializing containers for each allocator that wants to preserve my move construction optimization.

7.3.1 Example: Current `vector` implementations break for custom allocators

As a quick guide to the minimal example: * This examples assumes C++20[4] * `MyAlloc` is an allocator that derives from `std::allocator` to fulfil all basic allocator requirements, while providing its own `construct` call * `sharedptr<int>` is a standard library class with both a non-throwing move constructor and a copy constructor * The `report` lambda object simply displays the contents of the `vector` to ensure that the strong exception safety guatrantee is honored

The allocator supplied to the container is designed to throw after constructing a specific number of elements. The program is set up to ensure that number corresponds to creating one of the middle elements in a new vector buffer when an insertion at the back of the vector must reallocate.

The point being made is that `shared_ptr` satisfies the library requirements to demand the strong exception safety guarantee, but implementations simply look at the `shared_ptr` interface itself to conclude that they can safely move elements without creating a back-up, a la copy/swap idiom, despite the allocator's `construct` call causing the problem.

As long as an allocator does not supply its own `construct` call, the as-if interpretation means that vendors understand that a vector does not need to additionally protect itself against the concerns of a bad `construct`, and the straight `move` optimization is valid.

```
#include <iostream>
#include <memory>
#include <vector>
#include <utility>

template <class ELEM>
struct MyAlloc : std::allocator<ELEM> {
    inline static int count = 0;

    using std::allocator<ELEM>::allocator;

    template<class T, class... Args>
    void construct(T* p, Args&&... args) {
        if(8 == ++count) { throw count; }

        //std::cout << count << "\n";
        ::new(p) T(std::forward<Args...>(args...));
    }
};

int main(int argc, const char * argv[]) {
    using namespace std;

    vector<shared_ptr<int>, MyAlloc<shared_ptr<int>>> v;
    v.reserve(4);

    auto report = [&] {
        cout << "Report:\n";

        for (auto &elem : v) {
            cout << "\t" << elem << "\n";
        }
    };

    cout << "Fill:\n";

    for (int i = 0; i != 4; ++i) {
        v.emplace_back(make_shared<int>(i));
    }

    report();

    cout << "Grow:\n";
    try {
        v.emplace_back(make_shared<int>(4));
    }
    catch(int) {
    }

    report();

    cout << "Done:\n";
    return 0;
}
```

In practice, all known standard library implementations fail this test, including the Bloomberg BDE library, as they call the move constructor directly. Indeed, the library guarantees are written with respect to the move constructor, rather than the allocator call, almost as is this were the intended behavior.

Typical output from running the test example above:

```
Fill:
Report:
    0x559e8d8db318
    0x559e8d8db348
    0x559e8d8db378
    0x559e8d8db3a8
Grow:
Report:
    0x559e8d8db318
    0x559e8d8db348
    (nil)
    (nil)
Done:
```

Different implementations might move the elements in a different order, so the position of the empty shared pointers violating the strong exception safety guarantee will vary by Standard Library implementation.

7.4 Supporting trivial optimizations

The intent of paper [P2786R2] is to introduce into the Core language specification a new category of type that can be exploited to optimize relocation within data structures, including those in the C++ Standard Library. However, the preceding concerns also inhibit exploiting knowledge of trivial relocatability if a container's allocator supplies an implementation of `construct`. The following example demonstrates a simple failure case if trivial relocation is exploited, as the allocator cares about the address of the elements it constructs and destroys, but is not informed if the elements are moved into the new vector buffer by merely copying the bytes.

```
#include <cassert>
#include <iostream>
#include <memory>
#include <set>
#include <vector>


template <typename Element>
struct Telemark : std::allocator<Element> {
   std::set<void *> & d_owned;

   using alloctor::allocator;

   Telemark(std::set<void *> &telemetry) : d_owned{telemetry} {}

   template<class T, class... Args>
   void construct(T* p, Args&&... args) {
      if (!d_owned.emplace(p).second) {
         std::cout << "Overwrite at " << (void *)p << std::endl;
      }

      ::new((void*)p) T(std::forward<Args...>(args...));
   }


   template <class T>
   void destroy(T* p) {
      if (!d_owned.erase(p)) {
         std::cout << "Bad destroy at " << (void *)p << std::endl;
      }
   }
};

int main() {
    int a = 1;
    int b = 2;

    std::set<void *> registry;
    Telemark<int> alloc{registry};
```

```
    std::vector<int, Telemark<int>> v{alloc};
    v.reserve(4);
    assert(4 == v.capacity());

    auto fill = [&](int & i ) {
        v.clear();
        for (int j = 0; j != 4; ++j) {
            v.emplace_back(i);
        }
    };

    auto inject = [&](int & i) { v.emplace(v.begin() + 2, i); };

    auto report = [&] {
        for (auto& j : v) {
            std::cout << j;
        }

        std::cout << '\n';
    };

    for (int dummy : {1, 2}) {
        fill(a);
        inject(b);
        report();
    }
}
```

7.5 Relocation creates objects just like the `uninitialized` standard algorithms

While most algorithms do not care about constructing new objects, the `uninitialized_*` algorithms in the `<memory>` header are the counter-example. For consistency, the trivial relocation support added by [P2786R2] should be extended to support the object creation function templates in the Standard Library.

// vvvTHISvvv SHOULD BE REPHRASED AS AN EASE-OF-USE CONCERN

Similarly, for ease of use, a general purpose `relocate` function that uses move construction, and safely adopts trivial relocation optimizations where available, should be added to `<memory>` header.

8 Proposed Changes

We propose adding a new type trait as a user customization point to indicate types that have inconsistent construction/assignment behavior. This can be used as a hint where code is concerned with the difference. The minimal set of library components is then updated to forward this trait when needed.

We further propose adding a non-static member function to `allocator_traits` to support internal relocation/replacement that defaults to the current behavior when an allocator does not supply its own implementation, and the hint trait above produces `false_type`.

8.1 New type trait: `container_replace_with_assignment`

To properly address `std::vector<std::tuple<T &>>` we need a new trait to establish that a type has a different outcome for move-assignment than destroy/move-construction. To preserve backwards compatibility, we must assume this property defaults to supporting move-assignment, unless explicitly known to be `false` by a (partial) specialization of the new trait.

```
template <class T>
struct container_replace_with_assignment : is_move_assignable<T>::type {};

template <class T>
constexpr bool container_replace_with_assignment_v = container_replace_with_assignment{};
```

Similarly, this property does not hold for many compound types holding a member that does not have the assignment-is-construction property (such as a reference), e.g., `tuple`, `pair`, or `array`. Maybe `optional` and `variant`?

```
template <class ...TYPES>
struct container_replace_with_assignment<tuple<TYPES...>>
   : conjunction<container_replace_with_assignment<tuple<TYPES>>...>::type
```

```
{};

// also `pair`, `array`, funky views with reference semantics?
// --- play around with movable-box via `single_view`
```

8.2 New non-static member function: `allocator_traits::relocate`

If no `relocate` function is supplied by the allocator, the implicit definition for `allocator_traits::relocate` is:

— if `allocator_type` defines `relocate` member — call `allocator_type::relocate`

— otherwise if allocator_type defines at least one of `construct` and `destroy` — use `destroy`/`construct`-with-rvalue

— otherwise if the element type is trivially copyable — use `memmove`, (`trivial_relocate` once [P2786R2] lands)

— otherwise use move-assignment

This formula officially supports optimization for trivially copyable types (not going through `construct`). Otherwise, this revised definition has no change of behavior for `std::allocator`, which reaches last bullet.

— `std` container of `pmr` will work properly (semantic change) only with trivial relocation, [P2786R2], otherwise still old behavior from `std::allocator`

— `pmr` containers of `pmr` will get `pmr` optimization via new member, `polymorphic_allocator::relocate`

8.3 Preferred formula for `allocator_traits::relocate`

Finally, we reach our preferred formula for `allocator_traits::relocate`

```
if constexpr (requires requires{ allocator::relocate(...); }) {
    // forward to allocator::relocate
}
else if constexpr(requires requires{ allocator::construct(...); }) {
    // `allocator_traits::destroy(); allocator_traits::construct(rvalue)` each element
}
else if constexpr (is_trivially_relocatable_v<T>) {
    // trivially relocate
}
else if constexpr (container_replace_with_assignment_v<T>) {
    // move-assign elements
}
else {
    // destroy-move-construct elements
}
```

Note that in this formulation, the presence of `construct` and `destroy` trumps the element type itself being trivially relocatable. In such cases, we defer that optimization to the allocator itself providing a `relocate` member.

8.4 New *Cpp17Requirements* for internal relocation in a block container

…

8.5 Worked example

So let's apply this formula to `std::vector<std::pmr::vector<T>>`:

— `std::allocator` does not provide a `relocate` member

— `std::allocator` does not customize `construct` or `destroy`

— the element type `std::pmr::vector<T>` will be trivially relocatable in the future, but let's assume not for now to observe the default semantics

— `container_replace_with_assignment_v<std::pmr::vector<T>>` is `false`, as the vector has an `allocator_type` that does not propagate on move-assignment

— destroy-then-move each `std::pmr::vector<T>` to perform relocation

If we have a `pmr::vector<pmr::vector<T>>`, then we should pick up a new `relocate` member added to `std::pmr::polymorphic_allocator` as the first bullet, and that member will test and apply the trivial relocation optimization if `T` is trivially relocatable, or else fall back to construct/destroy:

```
void std::pmr::polymorphic_allocator::relocate(...) {
   if constexpr (is_trivially_relocatable_v<T>) {
      // trivially relocate
   }
   else if constexpr (container_replace_with_assignment_v<T>) {
      // move-assign elements
   }
   else {
      // `allocator_traits::destroy(); allocator_traits::construct(rvalue)` each element
   }
}
```

9 Optional Breaking Changes

In addition to the basic proposal above, we strongly believe that a small number of targeted fixes are important, even at the risk of changing the runtime behavior of valid C++11 programs. However, without these breaking changes the basic proposal remains an important non-breaking change that enables trivial relocation optimizations, and allows users to opt-in to containers correctly supporting their own types' semantics.

9.1 Fixing `vector<tuple<int&>>`

Returning to our motivating examples at the top of this paper, the behavior of a `tuple` containing a reference element is not changed by the basic proposal and will continue with the counter-intuitive behavior. The fix is to specialize `container_replace_with_assignment<`*`reference-type`*`>` as `false_type`.

By default, the assignment-is-construction property does not hold for reference types, even though the `is_move_assignable_v` trait is `true` for reference types. For example, a simple aggregate struct holding a single reference member is not move assignable either, and this is the behavior we expect from tuples as container *elements*.

```
template <class T>
struct container_replace_with_assignment : bool_constant<!is_reference_v<T>> {};
```

The issue is specific to `tuple`, `pair`, `array`, etc. holding a member reference, and would be addressed by changing the default for `container_replace_with_assignment<`*`reference-type`*`>`

```
template <class T>
struct container_replace_with_assignment : is_move_assignable<T>::type {};

template <class T>
constexpr bool container_replace_with_assignment_v = container_replace_with_assignment{};
```

// should be not-a-reference-type

9.2 Fixing element types using allocators that do not propagate on move assignment

Next, it does not hold for `pmr` containers themselves, or more generally, it does not hold for types that use an allocator that does not propagate on move-assignment. We may enable relocation when allocators are guaranteed to be always equal, but that may be a bad choice when thinking of relocating elements (with allocator telemetry) rather than values. By convention, types that use allocators do so by providing a type alias member named `allocator_type`.

To preserve maximum compatibility with existing containers and allocators, we further consider a type that has an allocator that always compares equal as-if it were propagating on move-assignment; this behavior can be changed for an allocator that does indeed compare about preserving allocators with elements, where such allocators store non-salient information that they wish to retain with the allocated element — so we might prefer to simplify the default to just `propagate_on_move_assignment`.

```
template <class T>
   requires requires { typename T::allocator_type; }
struct container_replace_with_assignment<T>
  : disjunction< allocator_traits<T::allocator_type>::propagate_on_move_assignment
               , allocator_traits<T::allocator_type>::allocators_always_equal>::type
{};
```

10 Addressing Objections

We anticipate several reasonable concerns from folks reviewing this paper. He we address the objections we anticipate, and future updates will add to this list as additional concerns are raised during review.

10.1 ABI breakage

In principle, there should be **no** ABI breakage, although there may be a change of behavior — as would occur with any ABI-compatible bug fix.

NOTE THAT THIS CLAIM IS YET TO BE REVIEWED BY THE ABI WORKING GROUP. WE ARE DEFERRING SENDING FOR AN OPINION UNTIL LEWG HAS EXPRESSED ENOUGH INTEREST FOR THE ABI GROUP TO SPEND TIME CONFIRMING OUR CLAIMS.

Specifically, nothing in this paper requires changing or adding data members for existing library products. All the changes arise from adding new traits, or creating types that were not previously expressible in the language, so there should be no impact on name-mangling, ensuring new and old code can link together, subject to the behavior change mentioned above (ODR violation?).

Readers with a long member may remember [P0181R1] by the same author made similar claims, yet was withdrawn from the C++17 FDIS due to the late discovery or surprise ABI breakage. Why is this case different?

The key difference is that that the default order proposal changed the specification for a default template argument, to use a metafunction that produced exactly the same result for all existing code. As the class templates always produced the same instantiation using exactly the same argument lists, the expectation was that the mangling of those types should be unaffected. This assumption was correct.

What had not been considered was that the mangling of the formula to produce the default argument was including in the mangling of function templates taking arguments of those container types. This additional mangling is not necessary and was a surprise to many. However, it does help an implementation diagnose undefined behavior when a template instantiation may be impacted by additional templates, such as whether a specialization for the `default_order` template might be seen by one TU, but not another. This ABI-check would catch cases that seemed to produce the same type, but for different reasons.

The library proposal before us has no such impact, even on default template arguments in the standard library. Any impact that would affect internal dispatch as a library implementation detail can be avoided by the use of `if constexpr` to perform any additional dispatching within the current ABI bounds. Hence, there should be no impact reminiscent of the older paper.

10.2 Breaking changes of behavior

This proposal could change the runtime behavior of existing programs. Is the proposed cure worse than the original problem? Here was address each known change of behavior for a currently valid program.

10.2.1 Really old code is generally not maintained so can be hard to audit

Note that there are exactly no changes of behavior for code written to the C++98 and C++03 standards. To enable the change of behavior, a type trait must be specialized by the user as different to the default. The few changes to programs written against a later standard are tightly targeted to containers of tuples where one of the elements is a reference type, and containers that use allocators that do not propagate on move assignment; we could remove those targeted changes and users would still benefit from the rest of this proposal.

10.2.2 Containers of `std::tuple` holding references

The proposal specializes the new `container_relocate_with_assignment` trait as `false_type` for any `tuple` containing an element that, in turn, specializes that trait as `false_type`. By definition, that set of types starts as empty, so no breakage would occur.

However, this proposal does extend that set of types beyond the empty set, as there is a real desire to fix current semantics. The initial set of types that specialize `container_relocate_with_assignment` as `false_type` is limited to all reference types; tuples that have no reference-type template arguments have no change of behavior, or ABI. For users who believe they are correctly depending on the current specification for this case, we refer to the second (motivating example)[#ex.vectorvvector], which demonstrates that the current behavior is surprisingly inconsistent and likely does not actually behave as users expect in all circumstance —acknowledging that determined users who consistently `reserve` space for their `vector` and never exceed that may be out there.

The second set of types that specialize `container_relocate_with_assignment` as `false_type` is types that have an `allocator_type` typedef-name for an allocator type that does not propagate on move-assignment. In the standard library, that set

is limited to containers using `std::pmr::polymorphic_allocator`. Note that `std::pmr::polymorphic_allocator` itself is not such a type, so tuples of allocators rather than containers have no change of behavior.

10.3 Potential for a negative impact on runtime performance

There are two aspects to this objection. First we observe that after this proposal, components like `std::pmr::vector<std::pmr::string>>` will suffer a significant performance penalty unless we also update `std::pmr::polymorphic_allcator` as proposed. The problem here is that, for a conforming implementation, that should be the expected behavior today. We retain the expected performance for `pmr` containers only because the current implementations are non-conforming. This paper proposes a simple way for those containers to maintain (or enhance) their performance, allowing vendors to fix their bugs. Once the vendors do fix their bugs, absent this paper, there is nothing clients can do to recover that performance.

The second category of reduced runtime performance is for container elements that specialize `container_replace_with_assignment` to `false_type`. That set is very small without users providing that specialization themselves, but as noted above, the most impacted set of users would be allocator authors who provide an allocator that does not propagate on move-assignment; objects using such an allocator must be destroy/construct-ed rather than assigned-to with the this proposal, unless the author updates their allocator to support the new `replace` member function. However, that code will now have the correct semantics for a container, and correctness is often preferable to performance. As stated, the user would have the ability to recover all that container performance, if appropriate, simply by adding on function to their allocator.

10.4 Container functions like `std::erase` retain the original problems

The key to proposed behavior change is that the elements that are managed by a container are a property of the container that is not exposed to the outside world, while can only inspect the values. This means the programs implemented using only the public "value" interface will continue to behave as they do today. For example, the container `erase` function uses the `remove` or `remove_if` standard algorithm to efficiently remove elements from a `vector` by first shuffling down the values, rather than trying to call `erase` on each element, where with a `std::list` the container function used the public member function of `list` to erase just the specified elements.

std::list

```
#include <iostream>
#include <list>
#include <tuple>

int main() {
    using Elem = std::tuple<int &>;

    int a[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    auto report_a = [&]() {
        for (auto const & i : a) {
            std::cout << i;
        }
        std::cout << '\n';
    };

    auto report = [](auto const & view) {
        for (auto const & i : view) {
            std::cout << std::get<0>(i);
        }
        std::cout << '\n';
    };

    report_a();                             //
        0123456789
    std::list<Elem> lst =
        {a[1],a[3],a[5],a[7],a[9] };
    report_a();                             //
        0123456789

    int value = 3;
    erase(lst, Elem(value));

    report(lst);                            //
        1579
```

std::vector

```
#include <iostream>
#include <tuple>
#include <vector>

int main() {
    using Elem = std::tuple<int &>;

    int a[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    auto report_a = [&]() {
        for (auto const & i : a) {
            std::cout << i;
        }
        std::cout << '\n';
    };

    auto report = [](auto const & view) {
        for (auto const & i : view) {
            std::cout << std::get<0>(i);
        }
        std::cout << '\n';
    };

    report_a();                             //
        0123456789
    std::vector<Elem> vec =
        {a[1],a[3],a[5],a[7],a[9] };
    report_a();                             //
        0123456789

    int value = 3;
    erase(vec, Elem(value));

    report(vec);                            //
        1579
```

std::list	std::vector
`    report_a(); //` `        0123456789` `    return 0;` `}`	`    report_a(); //` `        0125~~5~~47~~6~~89  ` return 0;` `}`

Despite not directly addressing the underlying problem in this case, we believe this proposal is still the right direction. Frequently users manipulate `vectors` directly, rather than eagerly adopting every new library API that we create. Giving users the ability to turn to the `vector` and obtain the desired behavior, even if less efficiently, is an important option that is not available today — where the surprising assignment behavior is forced upon users through both the algorithmic interface and calling members of `vector` directly.

If we were more ambitious we might propose adding an `if constexpr` test for the new `container_replace_with_assignment` trait in the `vector` implementation of the container erase function; instead, we prefer to minimize potential breakage of existing code, in the knowledge that users would now have the ability to fix their own code.

§ 11 Acknowledgements

Thanks to Michael Park for the pandoc-based framework used to transform this document's source from Markdown.

Thanks to ARthur O`Dwyer for inspiration for many of the awkward cases that arose while researching trivial relocatability.

Thanks to Lori Hughes for reviewing this paper and providing early editorial feedback.

§ 12 References

[LWG1102] Daniel Krügler. std::vector's reallocation policy still unclear.
https://wg21.link/lwg1102
[LWG2158] Nikolay Ivchenkov. Conditional copy/move in std::vector.
https://wg21.link/lwg2158
[LWG3308] Billy O'Neal III. vector and deque iterator erase invalidates elements even when no change occurs.
https://wg21.link/lwg3308
[LWG3758] Jiang An. Element-relocating operations of std::vector and std::deque should conditionally require Cpp17CopyInsertable in their preconditions.
https://wg21.link/lwg3758
[P0181R1] Alisdair Meredith. 2016-06-23. Ordered By Default.
https://wg21.link/p0181r1
[P2786R2] Mungo Gill, Alisdair Meredith. 2023-06-16. Trivial relocatability options.
https://wg21.link/p2786r2

---

1. C++23 adds `std::zip` based on the same property of `std::tuple`.↩

2. The output of this program is:

   ```
   11211aabaa
   22222aaaaa
   ```

   ↩

3. Prior to C++20, `std::allocator` had a `rebind` member template that was inherited by our custom allocators; for older language dialects, an additional `rebind` template must be supplied to hide that in the base to avoid bad behavior.↩

4. Prior to C++20, `std::allocator` had a `rebind` member template that was inherited by our custom allocators; for older language dialects, an additional `rebind` template must be supplied to hide that in the base to avoid bad behavior.↩